

The 80/10/10 Masking Recipe.

BERT masks 15% of tokens — but it does not just replace them with [MASK]. Of the chosen tokens, 80% become [MASK], 10% become a random token, and 10% are left untouched. This topic explains *why* that odd split exists, with the exact arithmetic for real sequence lengths — every count computed in full.

THE RECIPE

$$\Pr(\text{selected}) = 0.15, \quad \text{selected} \rightarrow \begin{cases} [\text{MASK}] & \text{with prob } 0.80 \\ \text{random token} & \text{with prob } 0.10 \\ \text{unchanged} & \text{with prob } 0.10 \end{cases}$$

Worked example. Exact expected counts for 128- and 512-token sequences; the train/inference mismatch made precise; a denoising / entropy view of the random-replacement branch.

Topic 2 of 3 in this module · companion to the course page · v1.0

Contents

1	What the recipe is	2
2	The train/inference mismatch	3
3	The denoising / entropy view	3
4	Exact expected counts	4
5	Properties / consequences that matter	4
6	Where this sits in the track	5
A	Reproducible (NumPy)	5

1 What the recipe is

For each training sequence, BERT picks 15% of token positions uniformly at random to be *prediction targets*. Crucially, being a target is decoupled from how the input is corrupted. Once a position is selected, the corruption applied to its input is drawn from a three-way split:

$$0.15 \xrightarrow{\text{of the selected}} \underbrace{0.80}_{\text{[MASK]}} \mid \underbrace{0.10}_{\text{random}} \mid \underbrace{0.10}_{\text{unchanged}} \quad (1)$$

In all three branches the model is still *scored* on predicting the original token. The difference is only in what the encoder *sees* at that position.

Branch	Input shown	Scored on
80%	[MASK]	original token
10%	a uniformly random vocabulary token	original token
10%	the original token (unchanged)	original token

The selection of targets and the corruption of inputs are two separate coin flips. Every selected position is graded on the same question — “what was the real token here?” — but the hint it is given ranges from *erased* ([MASK]), to *misleading* (a random token), to *fully visible* (unchanged).

2 The train/inference mismatch

The token [MASK] is an artefact of pretraining. At fine-tuning and inference time the input is clean text — [MASK] *never appears*. If BERT were trained with the naive recipe “selected → always [MASK]”, it would only ever be asked to produce contextual representations *at* [MASK] slots. The encoder could then take a shortcut: “build a rich representation only where I see [MASK]; everywhere else, just copy the token embedding through.” That policy scores perfectly in pretraining and collapses at fine-tuning, where there is no [MASK] to trigger the rich pathway.

$$\underbrace{P_{\text{train}}(\text{[MASK]}) = 0.12}_{0.15 \times 0.80} \neq \underbrace{P_{\text{infer}}(\text{[MASK]}) = 0}_{\text{clean text}} \quad (2)$$

2.1 How the 10/10 fix closes the gap

Keeping 10% *unchanged* and 10% *random* means the model cannot use the presence of [MASK] as the trigger to “think”:

- Because 10% of scored positions look like ordinary, unchanged tokens, the model must maintain a predictive representation for tokens that are *not* [MASK] — exactly the regime it faces at inference.
- Because 10% are replaced by a random token, the model cannot blindly trust the token it sees; any visible token *might* be corrupted, so it must reconcile each token against its context rather than copying it.

The model never knows, for a non- [MASK] position, whether it is (a) unselected, (b) selected-and-kept, or (c) selected-and-corrupted. To be safe it must build a genuine contextual representation of *every* token, which is exactly the behaviour fine-tuning needs.

The 80/10/10 split decorrelates “is this token going to be scored?” from “does this token look like [MASK]?”. That forces the encoder to compute high-quality representations at *all* positions, not just the masked ones — which is precisely why a pretrained BERT transfers to clean-text downstream tasks.

3 The denoising / entropy view

The 10% random-replacement branch quietly adds a second objective: *corruption detection*. When a position holds a random token that is inconsistent with its context, the model must notice the mismatch to recover the original — a mild denoising-autoencoder signal layered on top of pure cloze prediction.

The corruption itself is uniform over the $V = 30522$ vocabulary tokens, so each random replacement carries

$$-\log_2 \frac{1}{V} = \log_2 30522 \approx 14.9 \text{ bits}$$

of a-priori surprise, almost all of it resolvable only from context. And the three-way *branch* choice has entropy

$$H(0.80, 0.10, 0.10) = -(0.8 \log_2 0.8 + 0.1 \log_2 0.1 + 0.1 \log_2 0.1) = 0.9219 \text{ bits},$$

a small but nonzero uncertainty about *how* each scored slot was corrupted — enough to prevent the shortcut, not so much that the cloze signal is drowned out.

4 Exact expected counts

Treat the per-position selection as independent Bernoulli(0.15) draws; the expected number selected in a length- L sequence is $0.15L$, and each branch is that times its split fraction. For $L = 512$:

$$0.15 \times 512 = 76.8, \quad 0.80 \times 76.8 = 61.44, \quad 0.10 \times 76.8 = 7.68, \quad 0.10 \times 76.8 = 7.68.$$

The full table for the two standard sequence lengths:

Expected token counts per sequence					
L	selected (15%)	[MASK] (80%)	random (10%)	kept (10%)	untouched
128	19.20	15.36	1.920	1.920	108.80
512	76.80	61.44	7.680	7.680	435.20

As fractions of *all* tokens in the sequence (independent of L):

Fraction of every token's fate (intensity scaled to the max)				
	[MASK]	random	kept	untouched
share	0.1200	0.0150	0.0150	0.8500

(Check: $0.12 + 0.015 + 0.015 + 0.85 = 1.000$, and $0.12 + 0.015 + 0.015 = 0.15$ are the scored positions.)

5 Properties / consequences that matter

1. **Only 12% of tokens ever become [MASK]**, not 15% — the headline “15% masking” refers to scored positions, not [MASK] occurrences.
2. **All 15% contribute to the loss**, including the kept-unchanged 1.5% and the randomly-replaced 1.5%. The loss term from Topic 1 is summed over the full selected set $\mathcal{M}$.
3. **Counts are expectations.** Real implementations sample, so a given 512-sequence might have 74 or 80 selected; 76.8 is the mean.
4. **Sparse signal.** With only 15% of positions scored, BERT extracts learning from a small slice of each sequence — a key reason masked-LM pretraining is data-hungry compared to left-to-right language modelling, which scores 100% of positions.

The 10% “random token” is drawn from the *whole* vocabulary uniformly — it may even coincide with the original token by chance ($1/V \approx 3.3 \times 10^{-5}$). It is *not* restricted to “similar” tokens, and it is *not* drawn from the 80% [MASK] budget. Conflating “15%

masked” with “15% become [MASK]” is the single most common misreading of the recipe.

6 Where this sits in the track

Topic 1 built the loss $\mathcal{L}_{\text{MLM}} = -\sum_{i \in \mathcal{M}} \log \hat{p}_{i, x_i}$; this topic defines the set $\mathcal{M}$ and the corruption that produces the inputs x_i fed to that loss. Topic 3 adds the next-sentence-prediction objective and combines both into the full pretraining loss, then audits BERT-base’s parameter budget.

A Reproducible (NumPy)

Inputs: selection rate 0.15; split (0.80, 0.10, 0.10); lengths $L \in \{128, 512\}$; vocab $V = 30522$.
 Results: counts as tabulated; global shares (0.12, 0.015, 0.015, 0.85); branch entropy 0.9219 bits;
 $\log_2 V \approx 14.90$ bits.

```
import numpy as np
sel = 0.15
split = np.array([0.80, 0.10, 0.10]) # [MASK], random, unchanged
for L in (128, 512):
    n = sel * L # expected selected
    mask, rand, kept = n * split # branch counts
    print(L, n, mask, rand, kept, L - n)
# global shares of every token
shares = np.array([sel*0.8, sel*0.1, sel*0.1, 1-sel]) # 0.12 .015 .015 .85
H = -(split * np.log2(split)).sum() # 0.9219 bits
bits = np.log2(30522) # 14.8975 bits per random replacement
print(shares, shares.sum(), H, bits)
```